**РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ**

**Факультет искусственного интеллекта**

**Дисциплина: Инструменты обработки и визуализации данных**

# ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

**Студент: Князев Арсений Михайлович**

**Группа: ЗФИбд-01-24**

**Вариант № 7**

```
import os
import sys

from pyspark.sql import SparkSession

os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ.setdefault("SPARK_LOCAL_IP", "127.0.0.1")

spark = (
    SparkSession.builder
    .master("local[*]")
    .appName("TelcoChurnLab3")
    .config("spark.ui.showConsoleProgress", "false")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("ERROR")

print("spark:", spark.version)
```

```
spark: 4.0.1
```

Шаг 1 - загружаем данные и заполняем пропуски.

Шаг 2 - фильтруем клиентов по стажу и платежу.

Шаг 3 - строим сводную таблицу.

Шаг 4 - считаем ранг внутри каждого типа контракта.

Шаг 5 - добавляем категории стажа и масштабирование.

Шаг 6 - присоединяем справочник способов оплаты.

## Загрузка и очистка

```
import pandas as pd

from pyspark.ml.feature import Imputer
from pyspark.sql import Window
from pyspark.sql import functions as F
from pyspark.sql.functions import pandas_udf, udf
from pyspark.sql.types import DoubleType, StringType

DATA_FILE = "WA_Fn-UseC_-Telco-Customer-Churn.csv"

df_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(DATA_FILE)
)

print("строк:", df_raw.count(), "столбцов:", len(df_raw.columns))
df_raw.printSchema()
df_raw.show(5, truncate=False)
```

```
строк: 7043 столбцов: 21
root
 |-- customerID: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- SeniorCitizen: integer (nullable = true)
 |-- Partner: string (nullable = true)
 |-- Dependents: string (nullable = true)
 |-- tenure: integer (nullable = true)
 |-- PhoneService: string (nullable = true)
 |-- MultipleLines: string (nullable = true)
 |-- InternetService: string (nullable = true)
 |-- OnlineSecurity: string (nullable = true)
 |-- OnlineBackup: string (nullable = true)
 |-- DeviceProtection: string (nullable = true)
 |-- TechSupport: string (nullable = true)
 |-- StreamingTV: string (nullable = true)
 |-- StreamingMovies: string (nullable = true)
 |-- Contract: string (nullable = true)
 |-- PaperlessBilling: string (nullable = true)
 |-- PaymentMethod: string (nullable = true)
 |-- MonthlyCharges: double (nullable = true)
 |-- TotalCharges: string (nullable = true)
 |-- Churn: string (nullable = true)

+----------+------+-------------+-------+----------+------+------------+----------------+---------------+--------------+
|customerID|gender|SeniorCitizen|Partner|Dependents|tenure|PhoneService|MultipleLines   |InternetService|OnlineSecurity|
+----------+------+-------------+-------+----------+------+------------+----------------+---------------+--------------+
|7590-VHVEG|Female|0            |Yes    |No        |1     |No          |No phone service|DSL            |No            |
|5575-GNVDE|Male  |0            |No     |No        |34    |Yes         |No              |DSL            |Yes           |
|3668-QPYBK|Male  |0            |No     |No        |2     |Yes         |No              |DSL            |Yes           |
|7795-CFOCW|Male  |0            |No     |No        |45    |No          |No phone service|DSL            |Yes           |
|9237-HQITU|Female|0            |No     |No        |2     |Yes         |No              |Fiber optic    |No            |
+----------+------+-------------+-------+----------+------+------------+----------------+---------------+--------------+

+----------+------------+----------------+-----------+-----------+---------------+--------------+----------------+
|customerID|OnlineBackup|DeviceProtection|TechSupport|StreamingTV|StreamingMovies|Contract      |PaperlessBilling|
+----------+------------+----------------+-----------+-----------+---------------+--------------+----------------+
|7590-VHVEG|Yes         |No              |No         |No         |No             |Month-to-month|Yes             |
|5575-GNVDE|No          |Yes             |No         |No         |No             |One year      |No              |
|3668-QPYBK|Yes         |No              |No         |No         |No             |Month-to-month|Yes             |
|7795-CFOCW|No          |Yes             |Yes        |No         |No             |One year      |No              |
|9237-HQITU|No          |No              |No         |No         |No             |Month-to-month|Yes             |
+----------+------------+----------------+-----------+-----------+---------------+--------------+----------------+

+----------+--------------------------+--------------+------------+-----+
|customerID|PaymentMethod             |MonthlyCharges|TotalCharges|Churn|
+----------+--------------------------+--------------+------------+-----+
|7590-VHVEG|Electronic check          |29.85         |29.85       |No   |
|5575-GNVDE|Mailed check              |56.95         |1889.5      |No   |
|3668-QPYBK|Mailed check              |53.85         |108.15      |Yes  |
|7795-CFOCW|Bank transfer (automatic)|42.3          |1840.75     |No   |
|9237-HQITU|Electronic check          |70.7          |151.65      |Yes  |
+----------+--------------------------+--------------+------------+-----+
only showing top 5 rows
```

```
# пустые строки превращаем в null, затем заполняем медианой
df = df_raw.withColumn(
    "TotalCharges",
    F.when(F.trim("TotalCharges") == "", None)
    .otherwise(F.trim("TotalCharges"))
    .cast("double"),
)

missing_before = df.filter(F.col("TotalCharges").isNull()).count()

imputer = Imputer(
    inputCols=["TotalCharges"],
    outputCols=["TotalCharges_filled"],
    strategy="median",
)

df = (
    imputer.fit(df)
    .transform(df)
    .drop("TotalCharges")
    .withColumnRenamed("TotalCharges_filled", "TotalCharges")
)

string_columns = [name for name, dtype in df.dtypes if dtype == "string"]
df = df.fillna("Unknown", subset=string_columns)

print("пустых TotalCharges до обработки:", missing_before)
print("пустых TotalCharges после обработки:", df.filter(F.col("TotalCharges").isNull()).count())
df.select("customerID", "tenure", "MonthlyCharges", "TotalCharges", "Churn").show(8)
```

```
пустых TotalCharges до обработки: 11

пустых TotalCharges после обработки: 0
+----------+------+--------------+------------+-----+
|customerID|tenure|MonthlyCharges|TotalCharges|Churn|
+----------+------+--------------+------------+-----+
|7590-VHVEG|     1|         29.85|       29.85|   No|
|5575-GNVDE|    34|         56.95|      1889.5|   No|
|3668-QPYBK|     2|         53.85|      108.15|  Yes|
|7795-CFOCW|    45|          42.3|     1840.75|   No|
|9237-HQITU|     2|          70.7|      151.65|  Yes|
|9305-CDSKC|     8|         99.65|       820.5|  Yes|
|1452-KIOVK|    22|          89.1|      1949.4|   No|
|6713-OKOMC|    10|         29.75|       301.9|   No|
+----------+------+--------------+------------+-----+
only showing top 8 rows
```

## Фильтрация

```
filtered = df.filter(
    (F.col("tenure") > 12) & (F.col("MonthlyCharges") > 50.0)
)

print("клиентов после фильтра:", filtered.count())
filtered.select(
    "customerID", "tenure", "MonthlyCharges", "Contract", "Churn"
).show(10, truncate=False)
```

```
клиентов после фильтра: 3483
+----------+------+--------------+--------------+-----+
|customerID|tenure|MonthlyCharges|Contract      |Churn|
+----------+------+--------------+--------------+-----+
|5575-GNVDE|34    |56.95         |One year      |No   |
|1452-KIOVK|22    |89.1          |Month-to-month|No   |
|7892-POOKP|28    |104.8         |Month-to-month|Yes  |
|6388-TABGU|62    |56.15         |One year      |No   |
|8091-TTVAX|58    |100.35        |One year      |No   |
|0280-XJGEX|49    |103.7         |Month-to-month|Yes  |
|5129-JLPIS|25    |105.5         |Month-to-month|No   |
|3655-SNQYZ|69    |113.25        |Two year      |No   |
|9959-WOFKT|71    |106.7         |Two year      |No   |
|4183-MYFRB|21    |90.05         |Month-to-month|No   |
+----------+------+--------------+--------------+-----+
only showing top 10 rows
```

## Pivot

```
pivot_table = (
    df.groupBy("Contract")
    .pivot("InternetService", ["DSL", "Fiber optic", "No"])
    .agg(F.round(F.avg("MonthlyCharges"), 2))
    .orderBy("Contract")
)

pivot_table.show(truncate=False)
```

```
+--------------+-----+-----------+-----+
|Contract      |DSL  |Fiber optic|No   |
+--------------+-----+-----------+-----+
|Month-to-month|50.22|87.02      |20.41|
|One year      |61.4 |98.78      |20.82|
|Two year      |70.46|104.57     |21.78|
+--------------+-----+-----------+-----+
```

## Оконная функция

```
contract_window = Window.partitionBy("Contract").orderBy(
    F.col("MonthlyCharges").desc()
)

ranked = df.withColumn("price_rank", F.rank().over(contract_window))

ranked.filter(F.col("price_rank") <= 3).select(
    "Contract", "customerID", "MonthlyCharges", "price_rank"
).orderBy("Contract", "price_rank").show(20, truncate=False)
```

```
+--------------+----------+--------------+----------+
|Contract      |customerID|MonthlyCharges|price_rank|
+--------------+----------+--------------+----------+
|Month-to-month|2302-ANTDP|117.45        |1         |
|Month-to-month|8016-NCFVO|116.5         |2         |
|Month-to-month|9659-QEQSY|115.65        |3         |
|One year      |5734-EJKXG|118.6         |1         |
|One year      |8199-ZLLSA|118.35        |2         |
|One year      |2889-FPWRM|117.8         |3         |
|Two year      |7569-NMZYQ|118.75        |1         |
|Two year      |8984-HPEMB|118.65        |2         |
|Two year      |5989-AXPUC|118.6         |3         |
+--------------+----------+--------------+----------+
```

## UDF

```
def tenure_group(tenure):
    if tenure <= 12:
        return "Новичок"
    if tenure <= 36:
        return "Лояльный"
    return "Ветеран"


tenure_group_udf = udf(tenure_group, StringType())
df_with_group = df.withColumn("TenureGroup", tenure_group_udf("tenure"))

df_with_group.groupBy("TenureGroup").count().orderBy("TenureGroup").show()
```

```
+-----------+-----+
|TenureGroup|count|
+-----------+-----+
|    Ветеран| 3001|
|   Лояльный| 1856|
|    Новичок| 2186|
+-----------+-----+
```

```
monthly_stats = df.select(
    F.avg("MonthlyCharges").alias("mean"),
    F.stddev("MonthlyCharges").alias("std"),
).first()


@pandas_udf(DoubleType())
def scale_monthly_charges(values: pd.Series) -> pd.Series:
    return (values - monthly_stats["mean"]) / monthly_stats["std"]


df_scaled = df_with_group.withColumn(
    "MonthlyCharges_scaled",
    scale_monthly_charges("MonthlyCharges"),
)

df_scaled.select(
    "customerID", "MonthlyCharges", "MonthlyCharges_scaled"
).show(8)
```

```
+----------+--------------+---------------------+
|customerID|MonthlyCharges|MonthlyCharges_scaled|
+----------+--------------+---------------------+
|7590-VHVEG|         29.85|  -1.1602405389153658|
|5575-GNVDE|         56.95| -0.25961050958946186|
|3668-QPYBK|         53.85|   -0.362634608885045|
|7795-CFOCW|          42.3|   -0.746482462712137|
|9237-HQITU|          70.7|   0.1973512211570761|
|9305-CDSKC|         99.65|   1.1594633742561506|
|1452-KIOVK|          89.1|   0.8088491008469884|
|6713-OKOMC|         29.75|   -1.163563896957159|
+----------+--------------+---------------------+
only showing top 8 rows
```

## Join

```
payment_rows = [
    ("Electronic check", "Автоматический"),
    ("Mailed check", "Ручной"),
    ("Bank transfer (automatic)", "Автоматический"),
    ("Credit card (automatic)", "Автоматический"),
    ("Unknown", "Неизвестно"),
]

payment_reference = spark.createDataFrame(
    payment_rows,
    ["PaymentMethod", "Payment_Category"],
)

result = df_scaled.join(payment_reference, on="PaymentMethod", how="left")

result.select(
    "customerID",
    "PaymentMethod",
    "Payment_Category",
    "TenureGroup",
    "MonthlyCharges_scaled",
).show(12, truncate=False)
```

```
+----------+------------------------+---------------+-----------+--------------------+
|customerID|PaymentMethod           |Payment_Category|TenureGroup|MonthlyCharges_scaled|
+----------+------------------------+---------------+-----------+--------------------+
|1452-KIOVK|Credit card (automatic)  |Автоматический  |Лояльный   |0.8088491008469884   |
|7469-LKBCI|Credit card (automatic)  |Автоматический  |Лояльный   |-1.522486565470803   |
|8091-TTVAX|Credit card (automatic)  |Автоматический  |Ветеран    |1.1827268805487012   |
|5575-GNVDE|Mailed check             |Ручной          |Лояльный   |-0.25961050958946186 |
|3668-QPYBK|Mailed check             |Ручной          |Новичок    |-0.362634608885045   |
|6713-OKOMC|Mailed check             |Ручной          |Новичок    |-1.163563896957159   |
|9763-GRSKD|Mailed check             |Ручной          |Лояльный   |-0.4922455725149721  |
|7795-CFOCW|Bank transfer (automatic)|Автоматический  |Ветеран    |-0.746482462712137   |
|6388-TABGU|Bank transfer (automatic)|Автоматический  |Ветеран    |-0.286197373923806   |
|7590-VHVEG|Electronic check         |Автоматический  |Новичок    |-1.1602405389153658  |
|9237-HQITU|Electronic check         |Автоматический  |Новичок    |0.1973512211570761   |
|9305-CDSKC|Electronic check         |Автоматический  |Новичок    |1.1594633742561506   |
+----------+------------------------+---------------+-----------+--------------------+
only showing top 12 rows
```

## Вывод

Данные очищены, пустые значения `TotalCharges` заполнены медианой. Для клиентов построены фильтр, сводная таблица и рейтинг платежей внутри каждого типа контракта. Через UDF добавлена категория стажа, через pandas UDF - масштабированный платеж, после чего присоединён справочник способов оплаты.